

Modular Answer Sheet

Terrain • Site Boundary • OSM Buildings • Photogrammetry • Layer Controls

Student workflow: Start with a blank Cesium Sandcastle JavaScript panel. Paste Section 1 and run it. Append each following section beneath the existing code, running the application after every addition. Do not delete earlier sections.

Before you start

- Work in the JavaScript panel in Cesium Sandcastle.
- Replace **YOUR_CESIUM_ION_ACCESS_TOKEN** with the workshop token supplied by the instructor.
- The photogrammetry assets used in this lab are 4576801 and 4618928.
- If a checkpoint fails, inspect the browser console before continuing.
- Token:
 "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiIyYTRmMDgzNi0xYTVjLTQzYmUtYWVkbmNy1hOTE5OGFhYTM5O",
 WYiLCJpZCI6MjkwOTA3LCJpYXQiOiJlE3NDM3NzAxOTI9.0h5bZmhgMCUAcCmzqK74T8T05D7e_u6_a8DUBK7nGzg"

Section 1 • Add the Cesium ion access token

Outcome: Authenticates requests to Cesium ion.

```
// =====
// SECTION 1 - ACCESS TOKEN
// =====
Cesium.Ion.defaultAccessToken = "YOUR_CESIUM_ION_ACCESS_TOKEN";
console.log("Access token loaded");
```

Checkpoint: The console should display “Access token loaded”. The viewer is added in the next section.

Section 2 • Create the terrain-enabled viewer

Outcome: Creates the application viewer and configures terrain lighting.

```
// =====
// SECTION 2 - VIEWER SETUP
// =====
const viewer = new Cesium.Viewer("cesiumContainer", {
  terrain: Cesium.Terrain.fromWorldTerrain(),
  timeline: false,
  animation: false,
  infoBox: false,
  selectionIndicator: false
});

viewer.scene.globe.enableLighting = true;
viewer.scene.globe.depthTestAgainstTerrain = false;
console.log("Viewer initialised");
```

Checkpoint: The Cesium globe and world terrain should appear. The timeline, animation controls, information box and selection indicator should be hidden.

Section 3 • Add the site boundary

Outcome: Loads a GeoJSON polygon, displays its red outline at 625 metres and zooms to the site.

```
// =====  
// SECTION 3 - SITE BOUNDARY  
// =====  
const geojson = {  
  type: "FeatureCollection",  
  features: [{  
    type: "Feature",  
    geometry: {  
      type: "Polygon",  
      coordinates: [[  
        [11.18837119132374, 48.08428289693569],  
        [11.18837119132374, 48.082448398234845],  
        [11.198191497954355, 48.082448398234845],  
        [11.198191497954355, 48.08428289693569],  
        [11.18837119132374, 48.08428289693569]  
      ]]  
    }  
  }  
}  
};  
  
(async function () {  
  try {  
    const ds = await Cesium.GeoJsonDataSource.load(geojson, {  
      stroke: Cesium.Color.RED,  
      strokeWidth: 3,  
      fill: Cesium.Color.TRANSPARENT,  
      clampToGround: false  
    });  
  
    viewer.dataSources.add(ds);  
    for (const entity of ds.entities.values) {  
      if (entity.polygon) {  
        entity.polygon.height = 625;  
      }  
    }  
  
    await viewer.zoomTo(ds);  
    console.log("GeoJSON boundary added");  
  } catch (error) {  
    console.error("Boundary failed to load", error);  
  }  
})();
```

Checkpoint: A rectangular red site boundary should appear, and the camera should zoom to it.

Section 4 • Load OSM Buildings

Outcome: Loads Cesium OSM Buildings but keeps the layer hidden until the student enables it later.

```
// =====  
// SECTION 4 - OSM BUILDINGS  
// =====  
let osmBuildings = null;  
  
(async function () {  
  try {  
    osmBuildings = await Cesium.createOsmBuildingsAsync();  
    osmBuildings.show = false;  
    viewer.scene.primitives.add(osmBuildings);  
    console.log("OSM Buildings loaded");  
  } catch (error) {  
    console.error("Failed to load OSM Buildings", error);  
  }  
})();
```

Checkpoint: The console should report that OSM Buildings loaded. The buildings remain hidden by design.

Section 5 • Load the photogrammetry assets

Outcome: Loads both Cesium ion tilesets, applies the supplied adjustment to asset 4618928 and flies to their combined extent.

```
// =====  
// SECTION 5 - PHOTOGRAMMETRY  
// =====  
let photogrammetryTileset = null;  
  
(async function () {  
  try {  
    const assetIds = [4576801, 4618928];  
    const tilesets = [];  
  
    for (const id of assetIds) {  
      const tileset =  
        await Cesium.Cesium3DTileset.fromIonAssetId(id);  
  
      tileset.maximumScreenSpaceError = 2;  
      tileset.dynamicScreenSpaceError = false;  
      viewer.scene.primitives.add(tileset);  
  
      if (id === 4618928) {  
        photogrammetryTileset = tileset;  
        photogrammetryTileset.modelMatrix =  
          Cesium.Matrix4.fromTranslation(  
            new Cesium.Cartesian3(0, 0, 65)  
          );  
      }  
  
      tilesets.push(tileset);  
      console.log(`Tileset ${id} loaded`);  
    }  
  
    const combined = Cesium.BoundingSphere.fromBoundingSpheres(  
      tilesets.map(function (tileset) {  
        return tileset.boundingSphere;  
      })  
    );  
  
    viewer.camera.flyToBoundingSphere(combined, { duration: 0 });  
    console.log("All tilesets loaded");  
  } catch (error) {  
    console.error("Photogrammetry failed to load", error);  
  }  
})();
```

Checkpoint: Both tilesets should load and the camera should move to their combined area. Asset 4618928 is stored for the layer toggle.

Section 6 • Add interactive layer controls

Outcome: Creates a top-right panel for showing or hiding OSM Buildings and the designated photogrammetry layer.

```
// =====
// SECTION 6 - LAYER CONTROL WIDGET
// =====
const ui = document.createElement("div");
ui.style.position = "absolute";
ui.style.top = "10px";
ui.style.right = "10px";
ui.style.background = "rgba(42, 42, 42, 0.8)";
ui.style.padding = "10px";
ui.style.borderRadius = "4px";
ui.style.color = "white";
ui.style.fontFamily = "sans-serif";
ui.style.fontSize = "13px";
ui.style.display = "flex";
ui.style.flexDirection = "column";
ui.style.gap = "6px";
viewer.container.appendChild(ui);

// OSM Buildings toggle
const osmRow = document.createElement("label");
const osmCheckbox = document.createElement("input");
osmCheckbox.type = "checkbox";
osmCheckbox.style.marginRight = "6px";

osmCheckbox.addEventListener("change", function () {
  if (osmBuildings) {
    osmBuildings.show = osmCheckbox.checked;
    console.log("OSM Buildings:", osmCheckbox.checked);
  } else {
    console.warn("OSM Buildings are still loading");
  }
});

osmRow.appendChild(osmCheckbox);
osmRow.appendChild(document.createTextNode("OSM Buildings"));
ui.appendChild(osmRow);

// Photogrammetry toggle
const pgRow = document.createElement("label");
const pgCheckbox = document.createElement("input");
pgCheckbox.type = "checkbox";
pgCheckbox.checked = true;
pgCheckbox.style.marginRight = "6px";

pgCheckbox.addEventListener("change", function () {
  if (photogrammetryTileset) {
    photogrammetryTileset.show = pgCheckbox.checked;
    console.log("Photogrammetry:", pgCheckbox.checked);
  } else {
    console.warn("Photogrammetry is still loading");
  }
});

pgRow.appendChild(pgCheckbox);
pgRow.appendChild(document.createTextNode("Photogrammetry"));
ui.appendChild(pgRow);

console.log("Application loaded successfully");
```

Checkpoint: A layer panel should appear in the top-right. The OSM checkbox starts unticked; the photogrammetry

checkbox starts ticked. Test both controls.

Troubleshooting checklist

The viewer does not appear: Check that the access-token placeholder has been replaced and inspect the first console error.

The boundary is missing: Confirm that Section 3 was appended after the viewer code and that its coordinates were copied completely.

OSM Buildings are not visible: They are intentionally hidden at first. Add Section 6 and tick the OSM Buildings checkbox.

A photogrammetry asset fails: Confirm that the token can access assets 4576801 and 4618928.

A checkbox does nothing: Allow the related tileset to finish loading and then try the checkbox again.

The code contains `>`: Use the code in this answer sheet. It contains standard JavaScript arrows or function syntax rather than HTML-encoded characters.